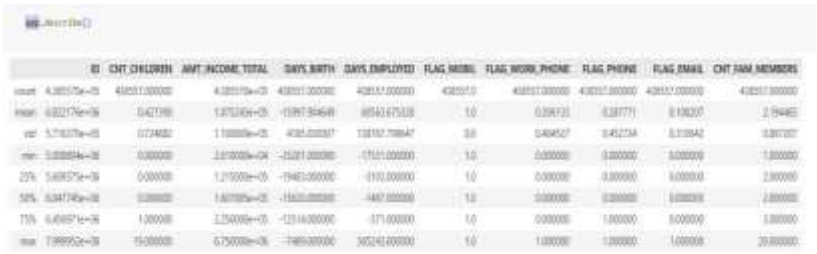


Data Collection and Preprocessing Phase

Date	24 June 2026
Team ID	SWTID-2026-5981
Project Title	Credit Card Approval Prediction
Maximum Marks	6 Marks

Data Exploration and Preprocessing Template

Data has been sourced from Kaggle. Dataset variables will be statistically analyzed to identify patterns and outliers, with Python employed for preprocessing tasks like normalization and feature engineering. Data cleaning will address missing values and outliers, ensuring quality for subsequent analysis and modeling, and forming a strong foundation for insights and predictions.

Section	Description
Data Overview	application_record.csv Dimension: 438557 rows x 13 columns Descriptive Statistics:  credit_record.csv Descriptive Analysis:

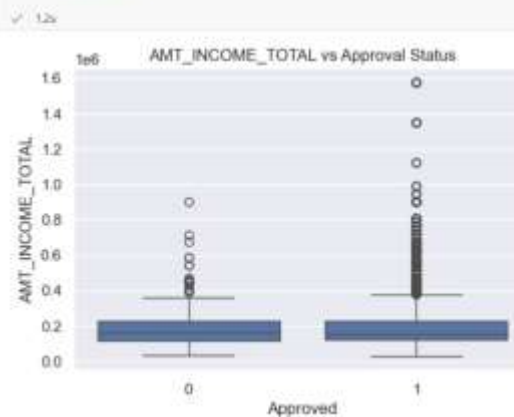
Bivariate Analysis

```
credit['default'] = credit['STATUS'].isin(['2', '3', '4', '5'])

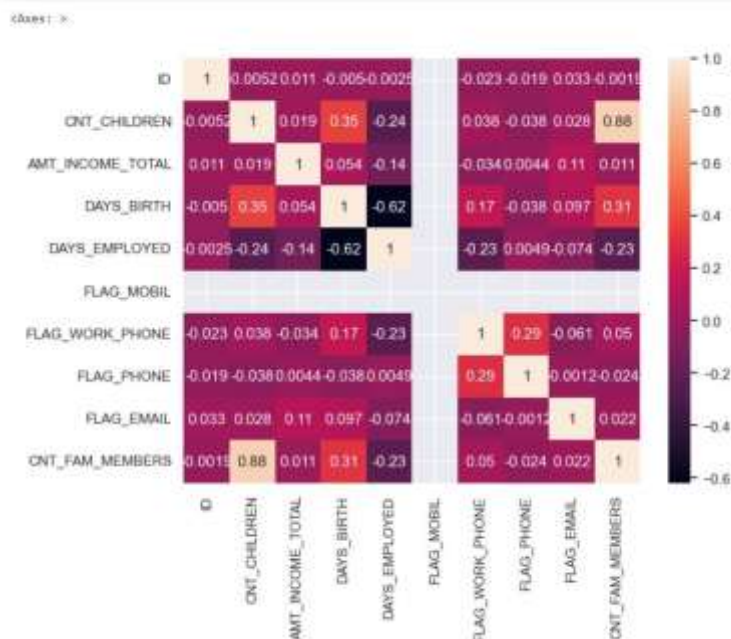
# Step 2: Group by ID, mark if any default
default_status = credit.groupby('ID')['default'].max().reset_index()
default_status['Approved'] = default_status['default'].apply(lambda x: 0 if x else 1)
default_status.drop(columns='default', inplace=True)

# Step 3: Merge with application data
app = app.merge(default_status, on='ID', how='inner')
numeric = ['AMT_INCOME_TOTAL', 'CNT_CHILDREN', 'CNT_FAM_MEMBERS', 'DAYS_BIRTH', 'DAYS_EMPLOYED']

for col in numeric:
    plt.figure(figsize=(8, 4))
    sns.boxplot(x='Approved', y=col, data=app)
    plt.title(f'{col} vs Approval Status')
    plt.show()
```



Multivariate Analysis



Outliers and Anomalies

Done using feature engineering and data cleaning stages as mentioned in notebook.

Data Preprocessing Code Screenshots

Loading Data

```

csv_path = "../application_record.csv" # update with actual file name
app = pd.read_csv(csv_path)
app.head()

```

MLTYPE	NAME_EDUCATION_TYPE	NAME_FAMILY_STATUS	NAME_HOUSING_TYPE	DAYS_BIRTH	DAYS_EMPLOYED	FLAG_MOBIL	FLAG_WORK_PHONE	FLAG_PHONE	FLAG_EMAIL
Working	Higher education	Cof marriage	Rented apartment	-1030	4042	1	1	0	0
Working	Higher education	Cof marriage	Rented apartment	-1030	4042	1	1	0	0
Working	Secondary / secondary special	Married	House / apartment	-21414	-7154	1	0	0	0
Associate	Secondary / secondary special	Single / not married	House / apartment	-6910	2024	1	0	1	1
Associate	Secondary / secondary special	Single / not married	House / apartment	-6910	2024	1	0	1	1

Handling Missing Data

```

D = app.isnull().sum()
D

```

ID	0
CODE_GENDER	0
FLAG_OWN_CAR	0
FLAG_OWN_REALTY	0
CNT_CHILDREN	0
AMT_INCOME_TOTAL	0
NAME_INCOME_TYPE	0
NAME_EDUCATION_TYPE	0
NAME_FAMILY_STATUS	0
NAME_HOUSING_TYPE	0
DAYS_BIRTH	0
DAYS_EMPLOYED	0
FLAG_MOBIL	0
FLAG_WORK_PHONE	0
FLAG_PHONE	0
FLAG_EMAIL	0
OCCUPATION_TYPE	134203
CNT_FAM_MEMBERS	0

dtype: int64

Data Transformation

```

def data_cleaning(data):
    # Adding number of family members with number of children to get overall family members.
    #data['CNT_FAM_MEMBERS'] = data['CNT_FAM_MEMBERS'] + data['CNT_CHILDREN']

    dropped_cols = ['FLAG_MOBIL', 'FLAG_WORK_PHONE', 'FLAG_PHONE',
                    'FLAG_EMAIL', 'OCCUPATION_TYPE', 'CNT_CHILDREN']
    data = data.drop(dropped_cols, axis = 1)

    #converting birth years and days employed to years.
    data['DAYS_BIRTH'] = np.abs(data['DAYS_BIRTH'])/365 #Absolute
    data['DAYS_EMPLOYED'] = data['DAYS_EMPLOYED']/365

    #Cleaning up categorical values to lower the count of dummy variables.
    housing_type = {'House / apartment': 'House / apartment',
                    'With parents': 'With parents',
                    'Participal apartment': 'House / apartment',
                    'Rented apartment': 'House / apartment',
                    'Office apartment': 'House / apartment',
                    'Co-op apartment': 'House / apartment'}

    income_type = {'Commercial associate': 'Working',
                   'State servant': 'Working',
                   'Working': 'Working',
                   'Pensioner': 'Pensioner',
                   'Student': 'Student'}

    education_type = {'Secondary / secondary special': 'secondary',
                       'Lower secondary': 'secondary',
                       'Higher education': 'Higher education',
                       'Incomplete higher': 'Higher education',
                       'Academic degree': 'Academic degree'}

    family_status = {'Single / not married': 'Single',
                     'Separated': 'Single',
                     'Widow': 'Single',
                     'Civil marriage': 'Married',
                     'Married': 'Married'}

    data['NAME_HOUSING_TYPE'] = data['NAME_HOUSING_TYPE'].map(housing_type)
    data['NAME_INCOME_TYPE'] = data['NAME_INCOME_TYPE'].map(income_type)
    data['NAME_EDUCATION_TYPE'] = data['NAME_EDUCATION_TYPE'].map(education_type)
    data['NAME_FAMILY_STATUS'] = data['NAME_FAMILY_STATUS'].map(family_status)

    return data
data = data_cleaning(app)
data.head()

```

Feature Engineering

Feature Engineering

```
def feature_engineering_target(data):
    good_or_bad = []

    for index, row in data.iterrows():
        paid_off = row['paid_off']
        over_1 = row['pastdue_1-29']
        over_30 = row['pastdue_30-59']
        over_60 = row['pastdue_60-89']
        over_90 = row['pastdue_90-119']
        over_120 = row['pastdue_120-149'] + row['pastdue_over_150']
        no_loan = row['no_loan']

        overall_pastdues = over_1 + over_30 + over_60 + over_90 + over_120

        if overall_pastdues == 0:
            if paid_off >= no_loan or paid_off <= no_loan:
                good_or_bad.append(1)
            elif paid_off == 0 and no_loan == 1:
                good_or_bad.append(1)

        elif overall_pastdues != 0:
            if paid_off > overall_pastdues:
                good_or_bad.append(1)
            elif paid_off <= overall_pastdues:
                good_or_bad.append(0)

        elif paid_off == 0 and no_loan != 0:
            if overall_pastdues <= no_loan or overall_pastdues >= no_loan:
                good_or_bad.append(0)
            else:
                good_or_bad.append(1)

    return good_or_bad
```

```
target = pd.DataFrame()
target['ID'] = pivot_tb.index
target['paid_off'] = pivot_tb['paid_off'].values
target['#_of_pastdues'] = pivot_tb['pastdue_1-29'].values + pivot_tb['pastdue_30-59'].values + \
    + pivot_tb['pastdue_60-89'].values + pivot_tb['pastdue_90-119'].values + \
    + pivot_tb['pastdue_120-149'].values + pivot_tb['pastdue_over_150'].values

target['no_loan'] = pivot_tb['no_loan'].values
target['target'] = feature_engineering_target(pivot_tb)
credit_app = data_cleansing(app).merge(target, how = 'inner', on = 'ID')
credit_app.drop('ID', axis = 1, inplace = True)
```

Save Processed Data

The processed data was saved dynamically in the app i.e. data frame for application_record.csv